

Demystifying AI

Pilot - UK AI Factory Antenna

Dr Adam Carter

Co-Lead, WP3 Training

EPCC

August 2026



EuroHPC
Joint Undertaking



Co-funded by
the European Union



Department for
Science, Innovation
& Technology



Pilot

UK AI Factory Antenna

| epcc |

UK National Supercomputing Centre



EuroHPC
Joint Undertaking



Co-funded by
the European Union



Department for
Science, Innovation
& Technology



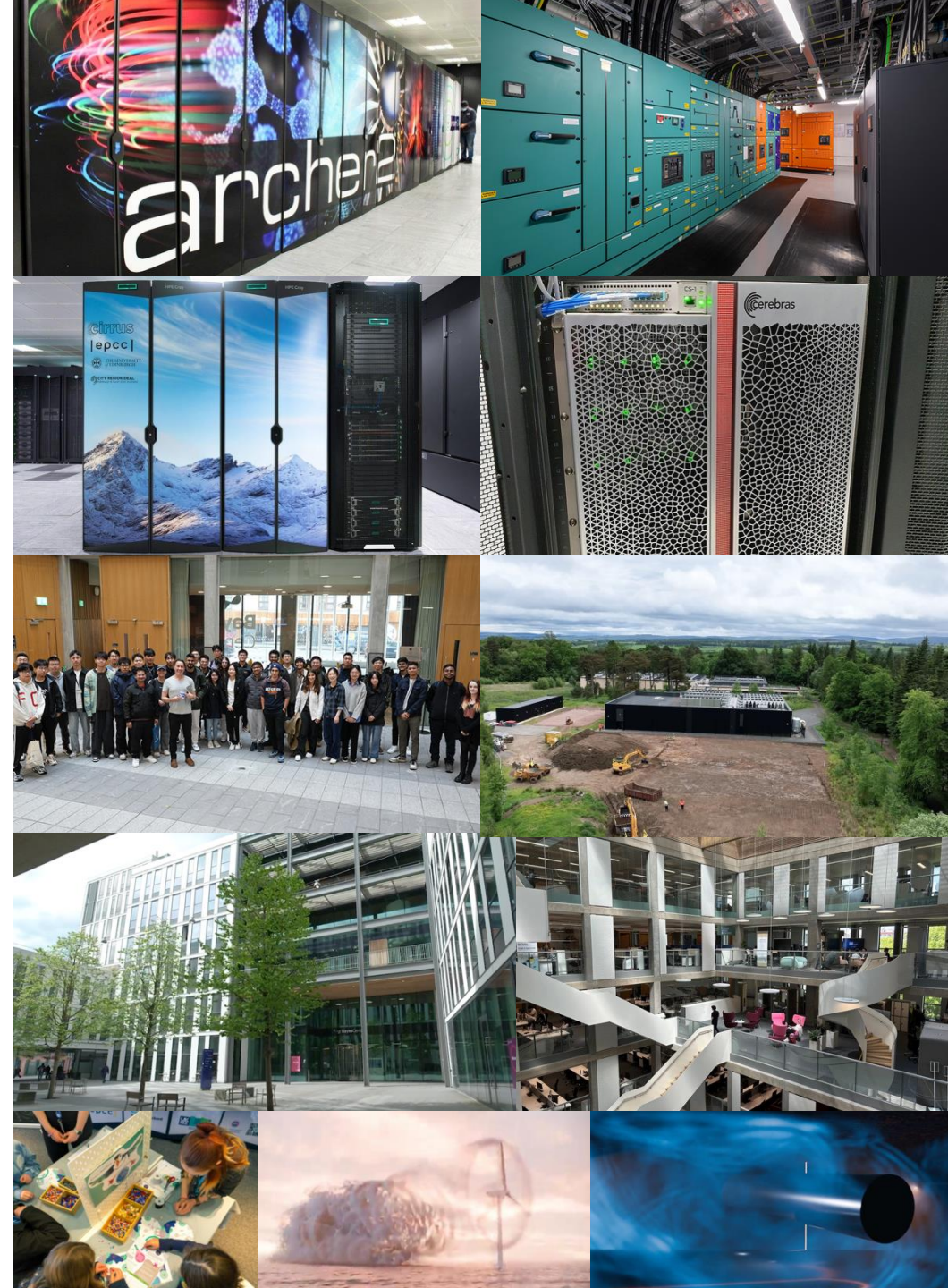
HammerHA

H L R I S

High Performance Computing Center | Stuttgart

Welcome!

- ...to EPCCC: home of the UK National Supercomputer Centre and the UK AI Factory Antenna, Pilot
 - Projects in many different areas of High Performance Computing, Data Science and AI with industry and academic partners throughout the UK, Europe and beyond
 - Running HPC & Data Services including ARCHER2, Cirrus and the Edinburgh International Data Facility
- We offer Masters Programmes, Postgraduate Professional Development courses and PhDs in High Performance Computing and Data Science
- We're in the Bayes Centre: The University of Edinburgh's hub for Data Driven Innovation



Timetable

09:30 **Welcome & Course Overview**

09:40 **What is AI? / Introduction to Machine Learning - Part 1**

10:30 *Break*

10:45 **Introduction to Machine Learning - Part 2**

11:35 **Introduction to Neural Networks**

12:30 *Lunch*

13:30 **Discussion: Ethical Considerations in AI**

14:00 **What happens when you enter a prompt into an AI assistant?**

15:00 *Break*

15:15 **Writing good prompts**

15:30 **Beyond the Prompt: Introduction to AI-assisted coding and Agentic AI**

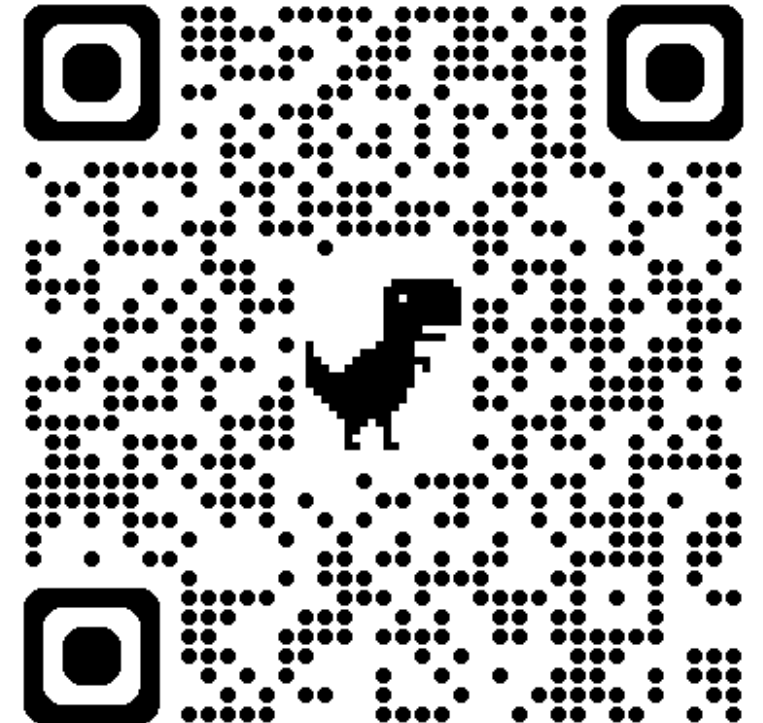
16:20 **Wrap-Up**

16:30 *Close*

Slides & Course Materials

- Access to course materials is available at:

<https://github.com/EPCCed/2026-08-21-Demystifying-AI>



What is AI?



What is AI?

- Getting a computer to act as if it is intelligent



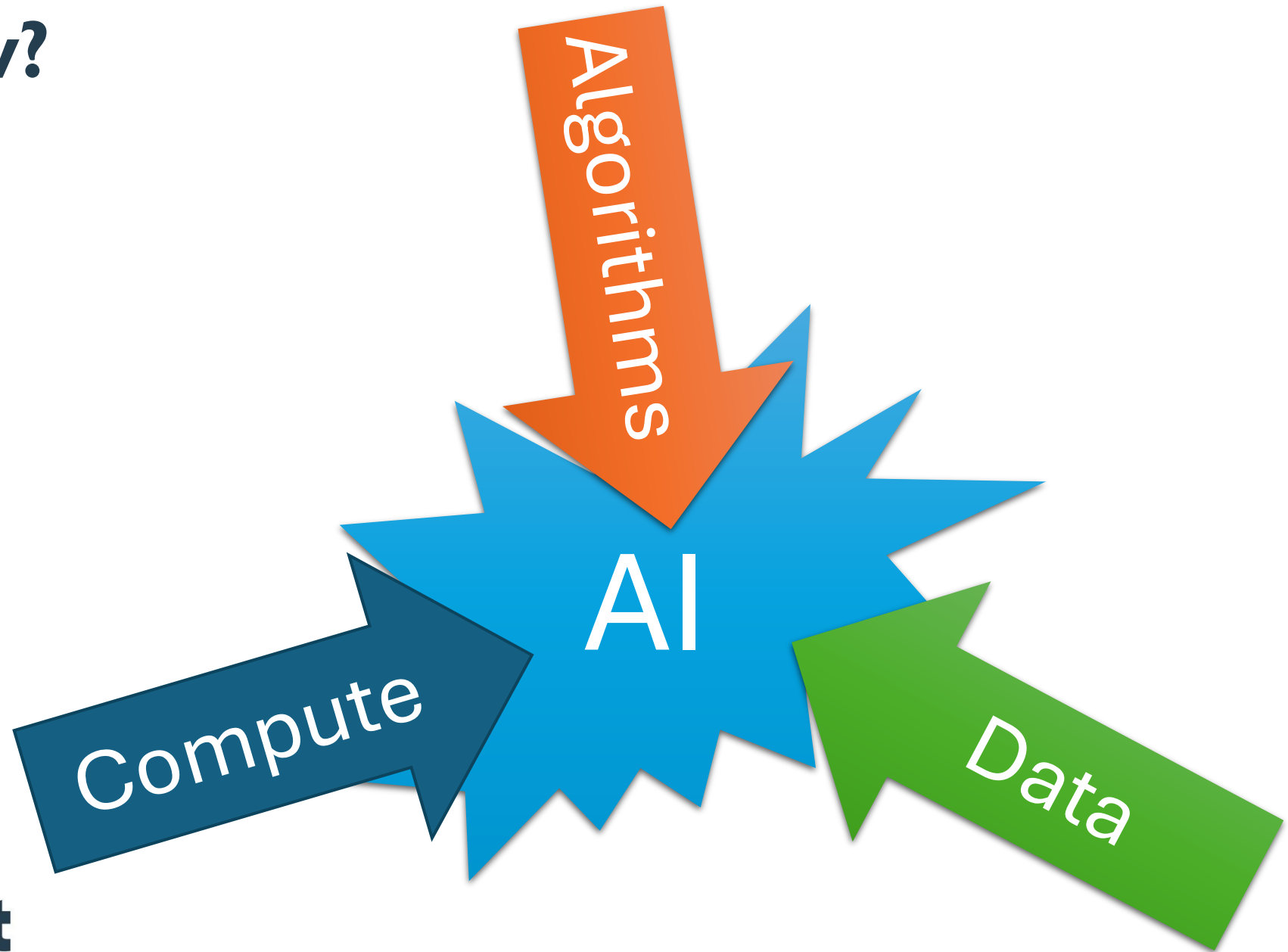
What is AI?

- “Artificial intelligence (AI) is the capability of computational systems to perform tasks typically associated with human intelligence, such as learning, reasoning, problem-solving, perception, and decision-making” -- wikipedia
- The term “Artificial Intelligence” is generally credited to **John McCarthy** in 1955: “Every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it.” And later, “Artificial intelligence is the science and engineering of making intelligent machines.”

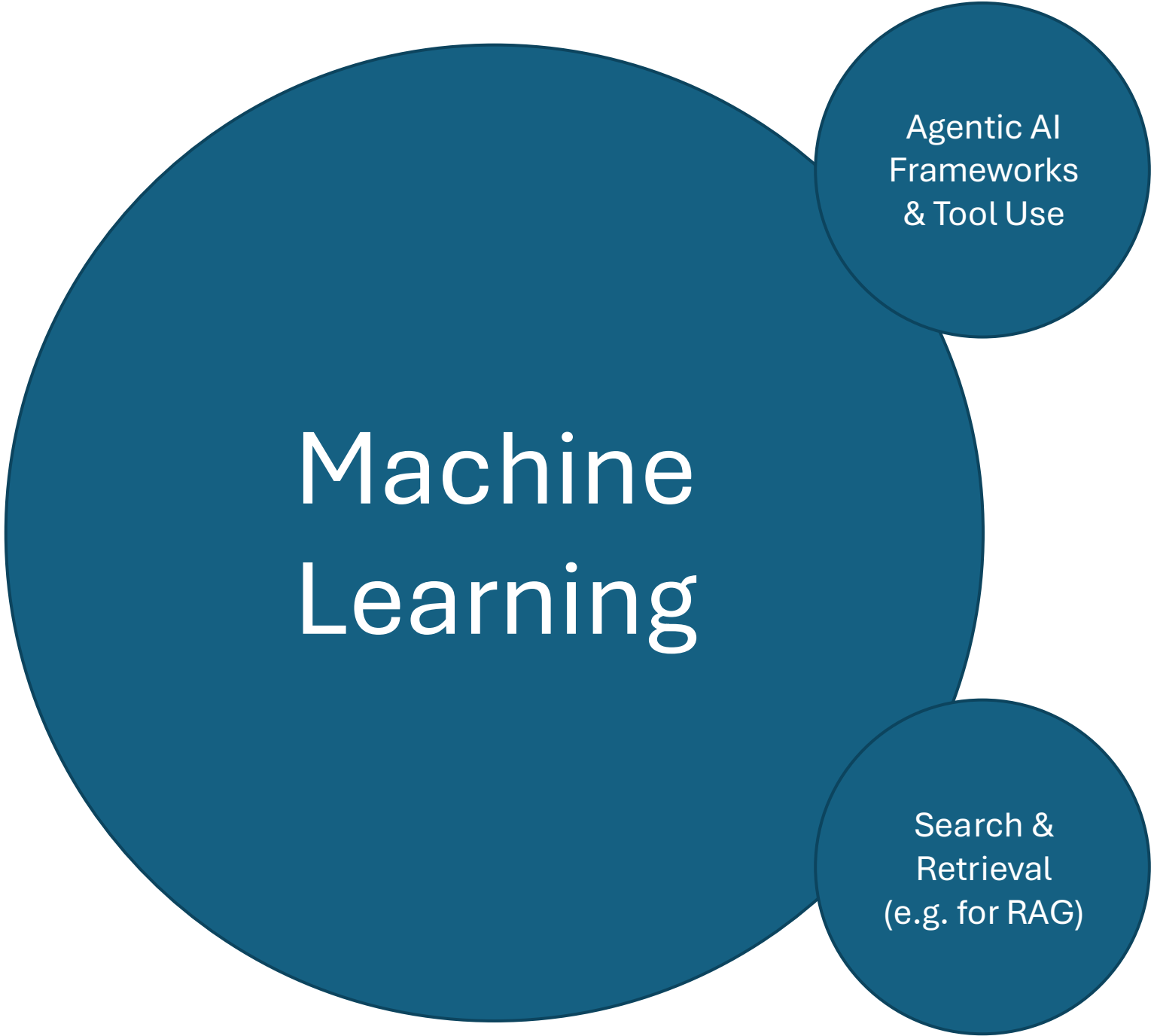
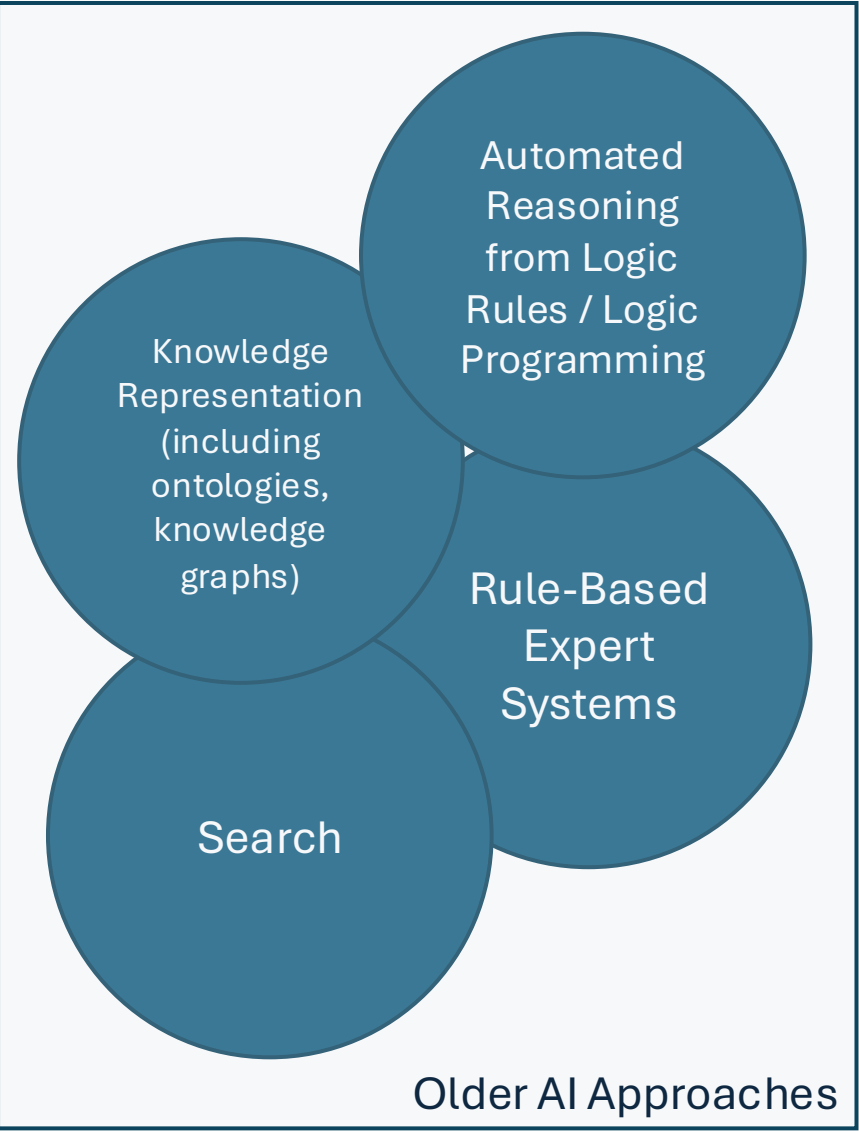
What is AI?

- “Artificial intelligence (AI) is technology that enables computers and machines to simulate human learning, comprehension, problem solving, decision making, creativity and autonomy.” – Cole Stryker, Eda Kavlakoglu, IBM
- “Artificial Intelligence (AI) can be defined in many ways. However, within this guidance, we define it as an umbrella term for a range of algorithm-based technologies that solve complex tasks by carrying out functions that previously required human thinking.” – UK ICO
- “Artificial intelligence (AI) is a broad category of software that can recognize patterns, learn from data, and produce useful outputs”. – OpenAI

Why now?

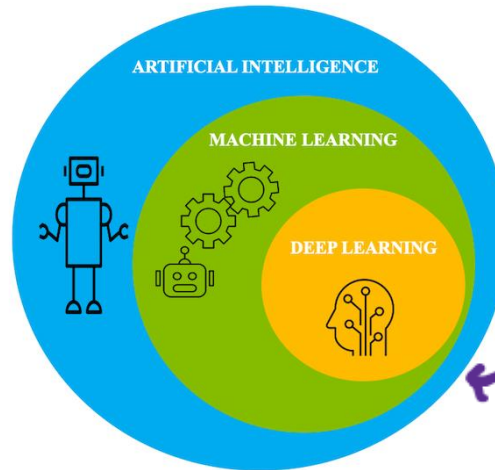
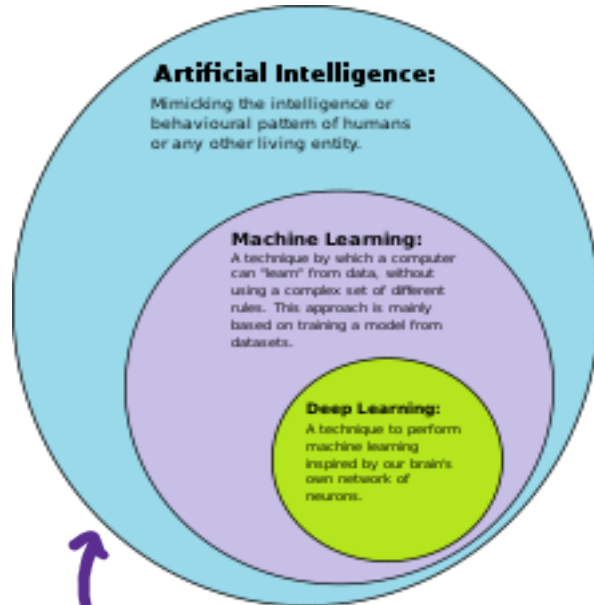
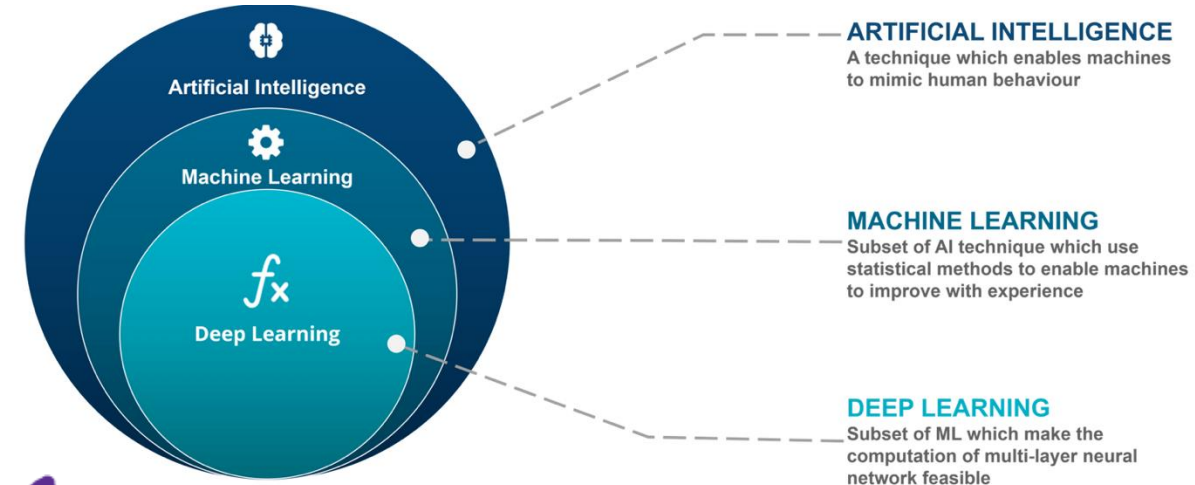
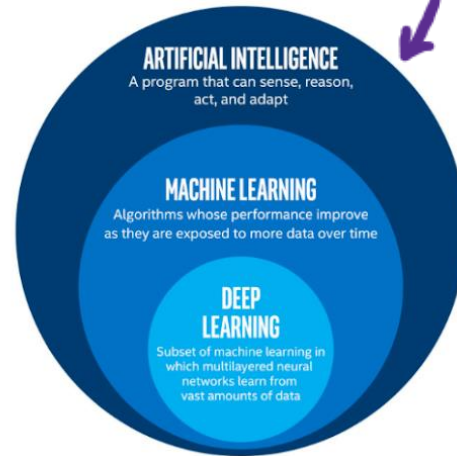


AI



Machine Learning, Deep Learning & AI

from <https://builtin.com/machine-learning/what-is-deep-learning>



from <https://www.edureka.co/blog/ai-vs-machine-learning-vs-deep-learning/>

from <https://developer.ibm.com/articles/an-introduction-to-deep-learning/>

from <https://commons.wikimedia.org/wiki/File:AI-ML-DL.svg>

Introduction to Machine Learning



What is Machine Learning?

- Getting a computer to do something, without explaining to the computer exactly how to do it
 - “Learn from these examples” – Supervised Learning
 - e.g., given many examples of pictures of cats and dogs with the label “cat” or “dog”, learn how to identify whether a new, unlabeled image is a cat or a dog
 - “Find patterns in this data” – Unsupervised Learning
 - e.g., given information about all your customers, can you group your customers into sets that are similar in some respect, perhaps so you could treat them as a group in a marketing campaign?
 - “Try to achieve this goal by learning from your experience” – Reinforcement Learning
 - e.g. get a robot to learn how to climb over a box by trying different movements of its legs to see which ones are most effective

Machine Learning

```
graph TD; ML[Machine Learning] --- UL[Unsupervised Learning]; ML --- SL[Supervised Learning]; ML --- RL[Reinforcement Learning];
```

Unsupervised Learning

Supervised Learning

Reinforcement Learning

Why is it so powerful?

- You can use (basically) the same algorithms for tasks in multiple domains, e.g.,
 - identifying genes that are similar to each other and identifying customers that are similar to each other
 - predicting which of your customers are likely to unsubscribe from your service and predicting the upcoming weather from current rain radar images
 - identifying tumours in medical images and identifying words spoken to a voice assistant
 - classifying images of cats and dogs, and classifying email as spam or not spam
 - teaching a robot to walk, and predicting the next word you will type in a text message
 - responding to a prompt in ChatGPT and creating an image from a text prompt

Some other definitions of machine learning

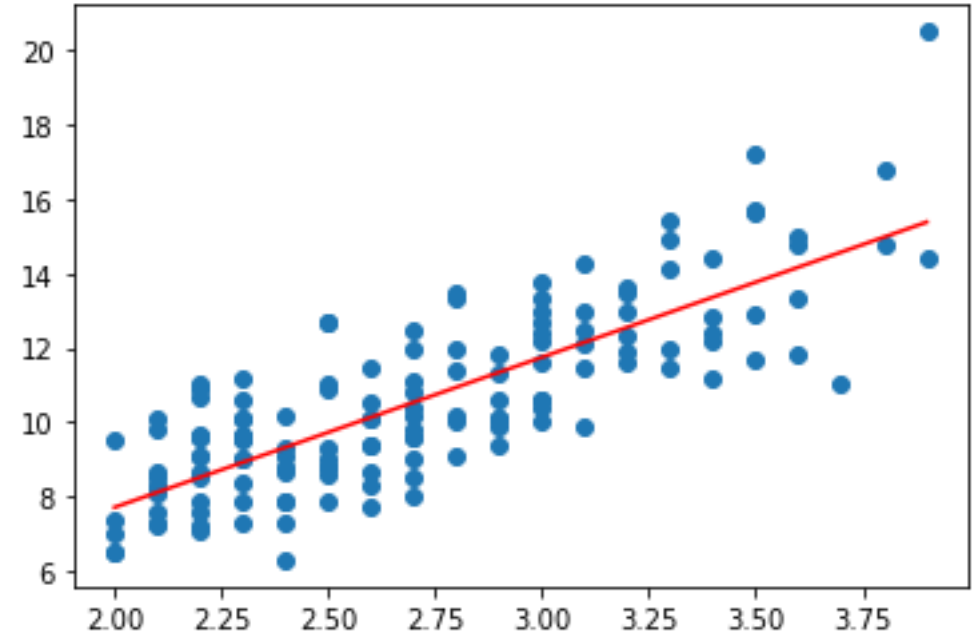
- “the field of study that gives computers the ability to learn without explicitly being programmed.” – Arthur Samuel, IBM (~1959)
 - This is possibly a paraphrase, but it is widely accepted that the term was coined by Arthur Samuel
- “Machine learning (ML) is a field of study in artificial intelligence concerned with the development and study of statistical algorithms that can effectively generalize and thus perform tasks without explicit instructions” – Wikipedia (accessed November, 2023)
- “Machine learning is a branch of artificial intelligence (AI) and computer science which focuses on the use of data and algorithms to imitate the way that humans learn, gradually improving its accuracy.” – IBM
- “it's the science of getting computers to learn without being explicitly programmed” – Andrew Ng, Stanford/Coursera/...

What do the machines learn?

- Central to Machine Learning (ML) is the idea of a **model**
- A model is learned from data using a **machine learning algorithm** and the model can then be used to make predictions/classifications from new, previously unseen data (in the case of supervised learning) or to cluster/group the data (in the case of unsupervised learning) or to decide on a course of action in an observed environment (reinforcement learning)
- Generally speaking, with ML, machines do *not* learn facts or build an understanding the way humans do; they learn models/patterns
 - No reason why this isn't possible in future, but that's not how ML currently works, even in ChatGPT!

Linear Regression as an example of ML

- Given a new value of data point with known x but unknown y can you predict y ?
- The **model** here is the red line
- The model is **learned** using a process which finds the best fit for the existing datapoints: the “**training data**”
- We sometimes say we **fit** the model to the training data, or that we **train** the model using the training data
- Key point: In ML, like here, we often decide on the shape of the model (here a straight line) but we let the machine learning algorithm find what the *best* straight line is...



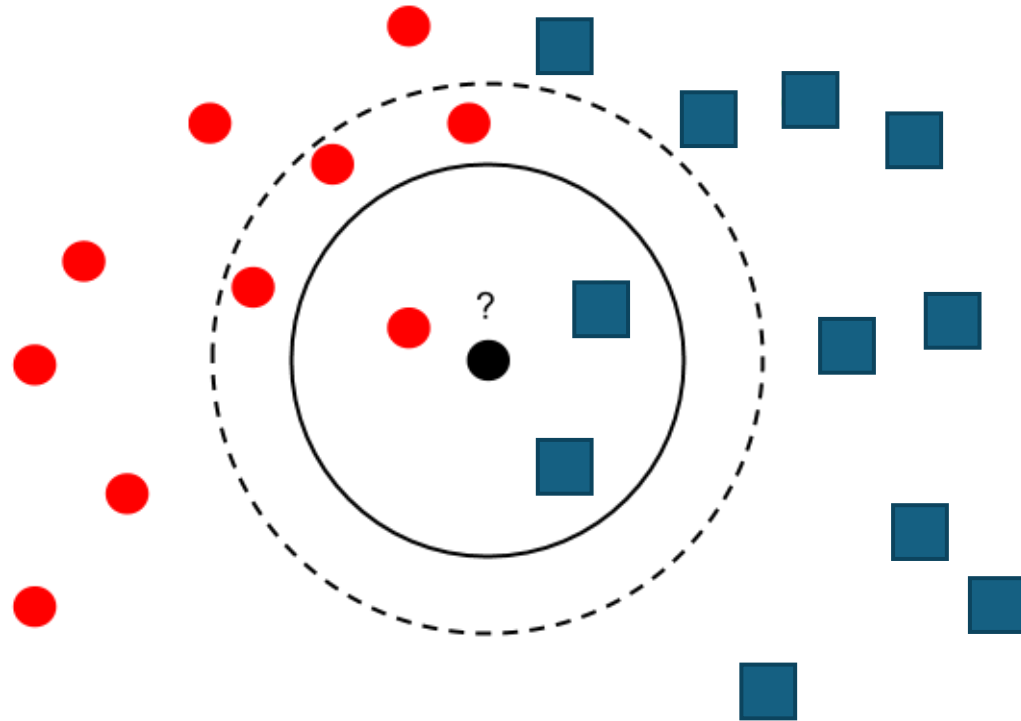
PRACTICAL: Linear Regression

- Connect to <https://notebook.eidf.ac.uk> in your web browser. Log in using the account you created when you registered for the course.
- Open a new (blank) Python notebook
- Type the following in the first cell to get the course files:
`!git clone https://github.com/EPCCed/2026-08-21-Demystifying-AI`
- Use the sidebar to navigate into the folder
`2026-08-21-Demystifying-AI/practicals`
- Open LinearRegression.ipynb and step through the cells

Break

- Course resumes at 10:45

k-Nearest Neighbours as an example of ML

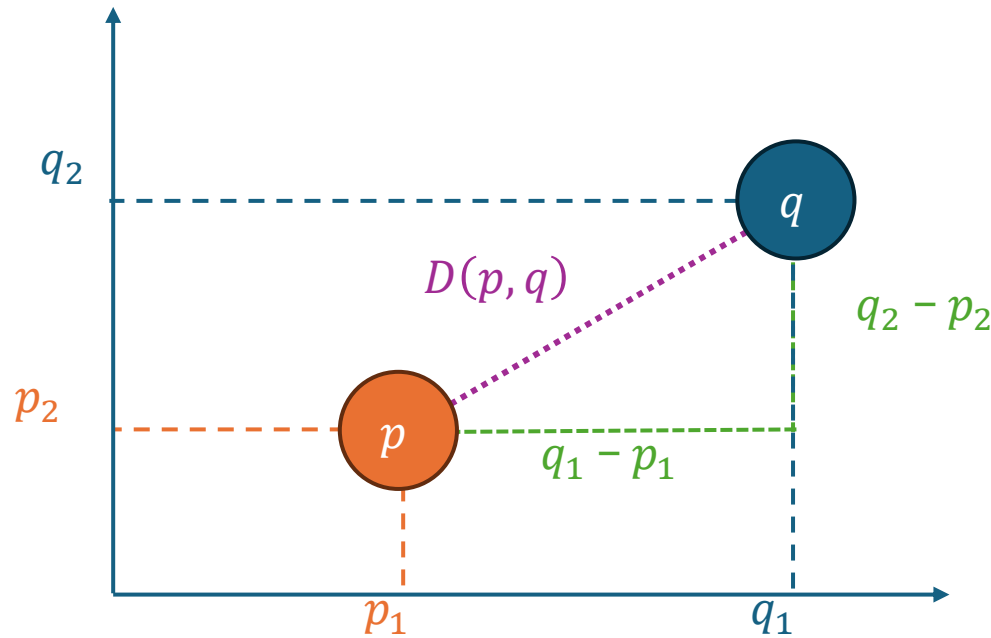


Two decisions to make:

- How to define similarity or closeness
- How many neighbours (k) should vote

Basic definition of similarity

- **Euclidean distance** works for real valued features
 - $D(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$



Basic definition of similarity

- **Euclidean distance** works for real valued features

- $D(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$

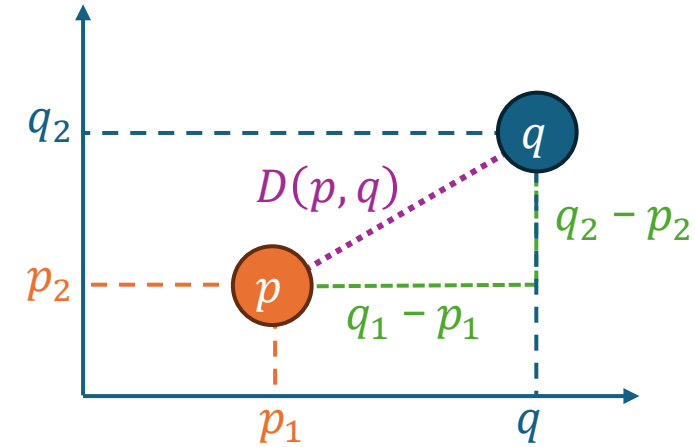
- Consider (age, salary) feature pair

- Euclidean distance measure is heavily biased towards salary
 - Normalise data first

- $x_{(i)} \mapsto \frac{x_{(i)} - \bar{x}}{\sigma_x}$

Here, x denotes a feature (e.g., “age”, “salary”) and i labels the row

age	salary
25	34,675
27	42,534
56	41,345



Basic definition of similarity

- **Euclidean distance** works for real valued features

- $D(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$

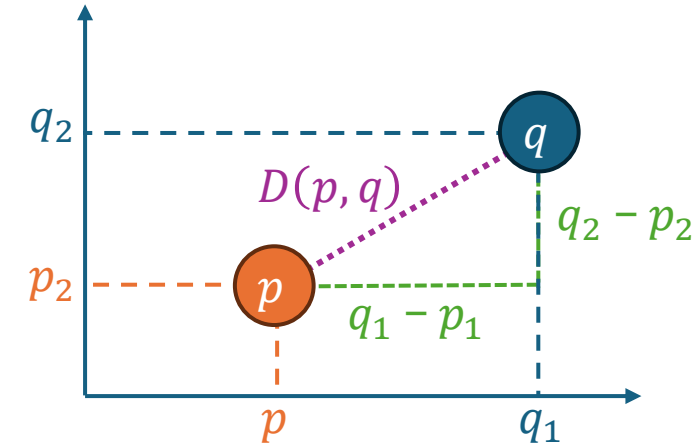
- Consider (age, salary) feature pair

- Euclidean distance measure is heavily biased towards salary

- Normalise data first

- $x_i \mapsto \frac{x_i - \bar{x}}{\sigma_x}$

- Avoid highly **correlated** features as these give extra weight to that feature

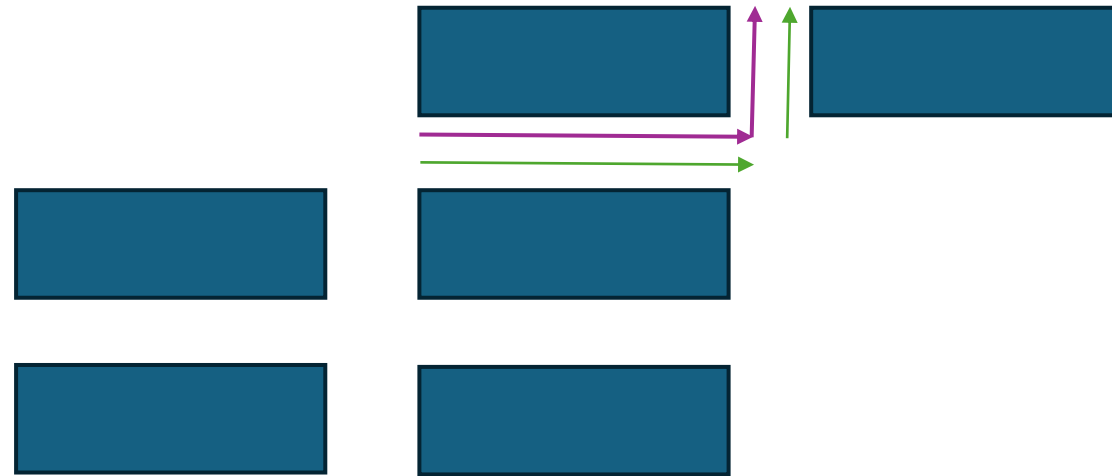


age	salary
25	34,675
27	42,534
56	41,345

age	years of work	salary
25	3	34,675
27	5	42,534
56	33	41,345

Some other measures: Manhattan Distance

- $D_{\text{Manhattan}}(x, y) = \sum_{i=1}^n |y_i - x_i|$



Alternative distance functions I

- Jaccard Similarity

- How similar two sets are

- $A = \{\text{Sheep, Cow, Hen}\}$, $B = \{\text{Cow, Horse, Hen}\}$

$$J(A, B) = \frac{A \cap B}{A \cup B}$$

- Cosine similarity

- Measure of angle between vectors
 - 1 identical, -1 exactly opposite, 0 independent
 - Used in text analysis and large language models
 - Can be used to classify documents, social media posts etc.:
("I love", "I hate", "delighted", "broken", "not", ...)
"I love my new tablet" -> (1,0,1,0,0,...)
"I hate my broken tablet. Not working. Not happy" -> (0,1,0,1,2,...)

$$\cos(x, y) = \frac{x \cdot y}{\|x\| \|y\|}$$

Alternative distance functions 2

- Mahalanobis Distance

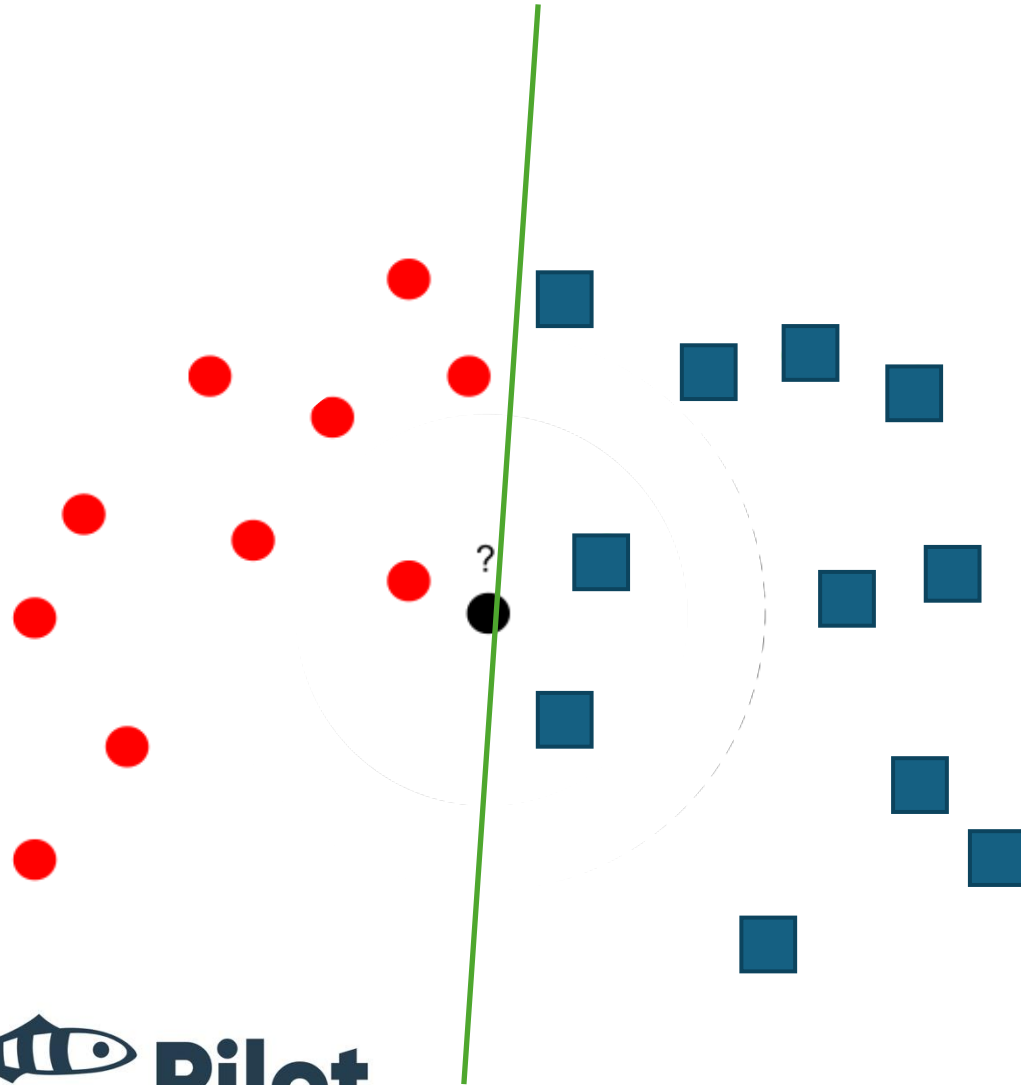
- Real valued vectors
- Scale invariant
- Takes correlation into account
- No need to normalise (effectively by co-variance (S) matrix)

$$D(x, y) = \sqrt{(x - y)^T S^{-1} (x - y)}$$

- Hamming Distance

- Distance between two sequences of same length
- Simply count the differences at each point in the sequence
- $D(\text{"GGATC"}, \text{"GAATT"}) = 2$
- Used in biology for simple matching of short sequences

An alternative classification approach



Goal:

Find a **decision boundary** and use this as the **model** to classify new instances.

This is another **parameterised model**.

Here the parameters are, again, a slope and an intercept.

Training

Training Set (Model)

5.1, 1.4, setosa
5.7, 1.3, setosa
...
6.3, 6.0, virginica

Model Architecture

e.g. $\beta_0 + \beta_1 x_1 = 0$

Definition of similarity

e.g. Euclidian distance

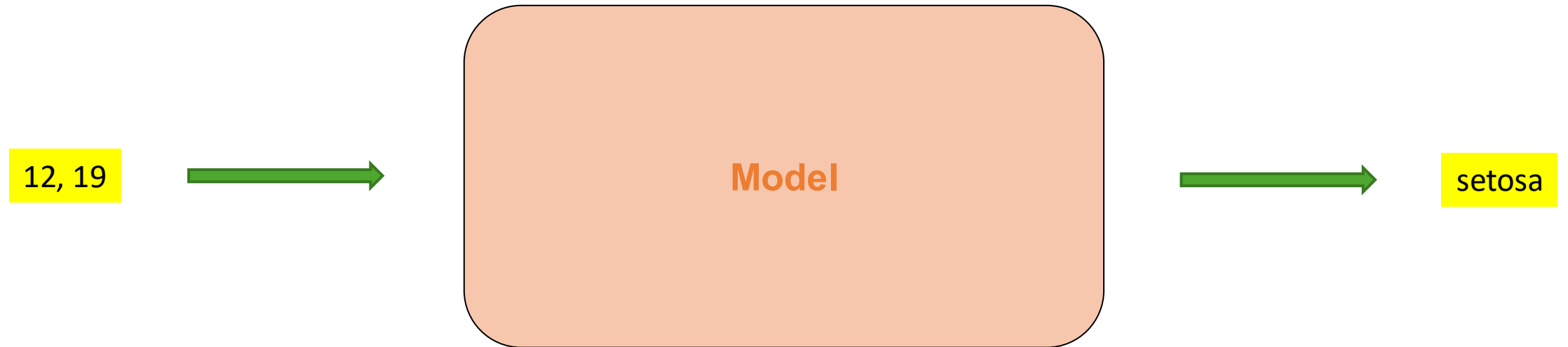
Hyperparameters

e.g. $k=3$

Model



Inference



Some important ideas

- Models are only as good as the data they're trained on
 - Possibility for bias, changing underlying conditions
 - Rare events can be harder to model (less training data)
- Machine learning models are often opaque:
 - It can be difficult to “understand” why they make a certain prediction
- There are different ways of deciding if a model is “good”
 - Would you prefer your AI poisonous mushroom predictor to be 98% accurate, or would you prefer that it was only accurate 95% of the time but erred on the “safe” side when unsure?
- Training models is often computationally expensive
 - Although modern computing resources make this more accessible than ever
 - Training is usually much more resource intensive than prediction/inference

PRACTICAL: Classification

- Return to the notebook server
- Open kNN.ipynb and step through the cells

Intro to k-Nearest Neighbours

In this practical we'll look at **classification**, a type of **supervised learning** that predicts which class a data point will belong to. So, in contrast to regression, which predicts a continuous value, this type of machine learning predicts from a number of pre-defined classes.

We'll start by loading some libraries. We're going to use **scikit-learn** both to create some synthetic data, and to do the classification. We'll also use **matplotlib** for plotting.

In [1]:

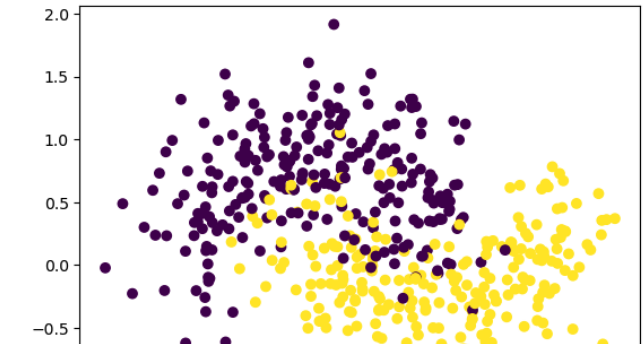
```
import sklearn as sk
import sklearn.datasets as skd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
```

In [2]:

```
X,y = skd.make_moons(n_samples=500,noise=0.3, random_state=1)
plt.scatter(X[:,0], X[:,1], c=y)
```

Out[2]:

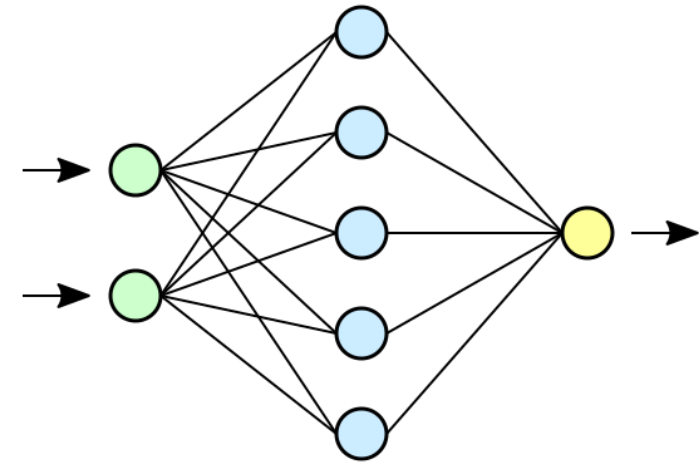
<matplotlib.collections.PathCollection at 0x751e362df8c0>



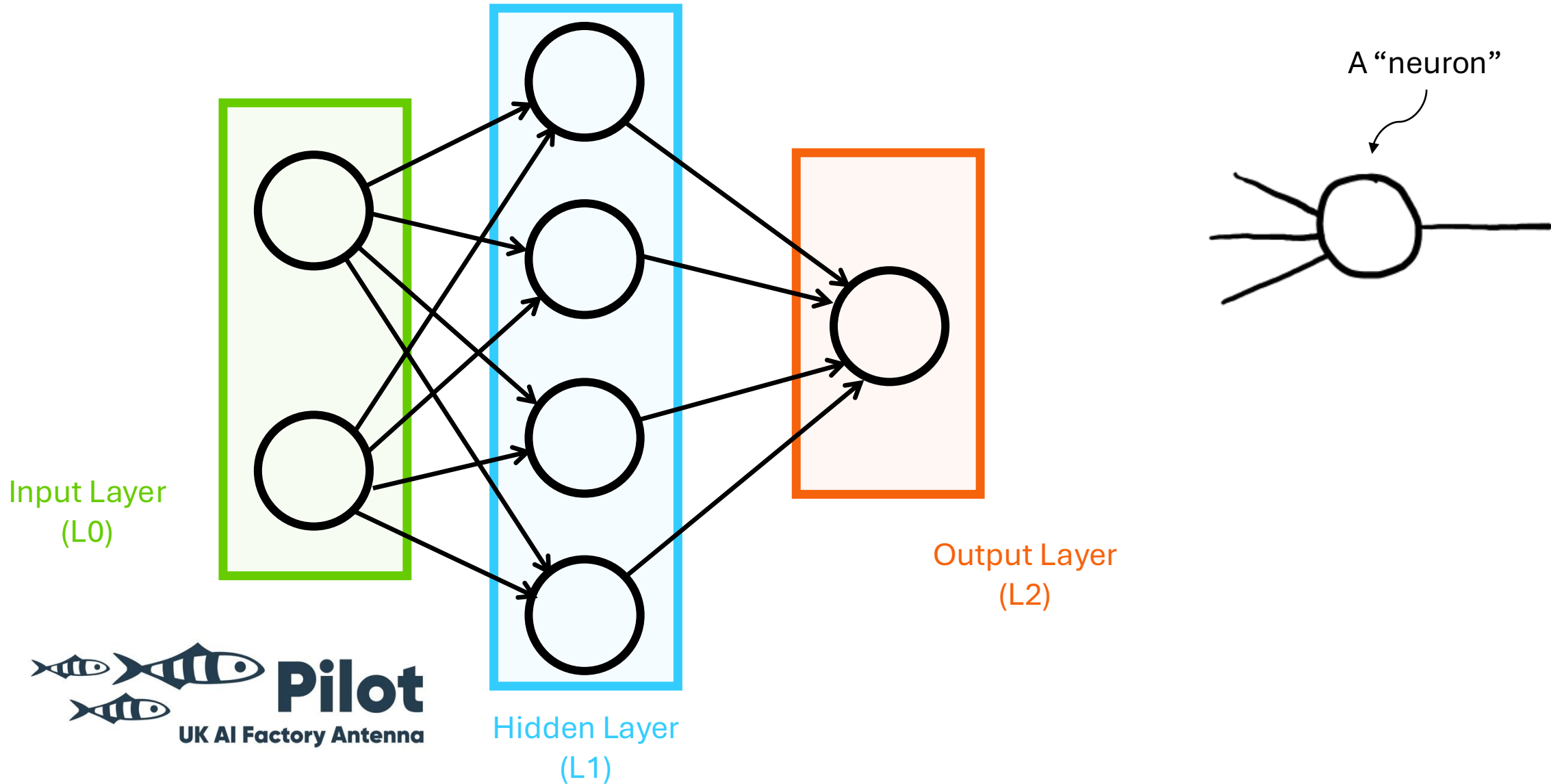
Introduction to Neural Networks

What is a Neural Network?

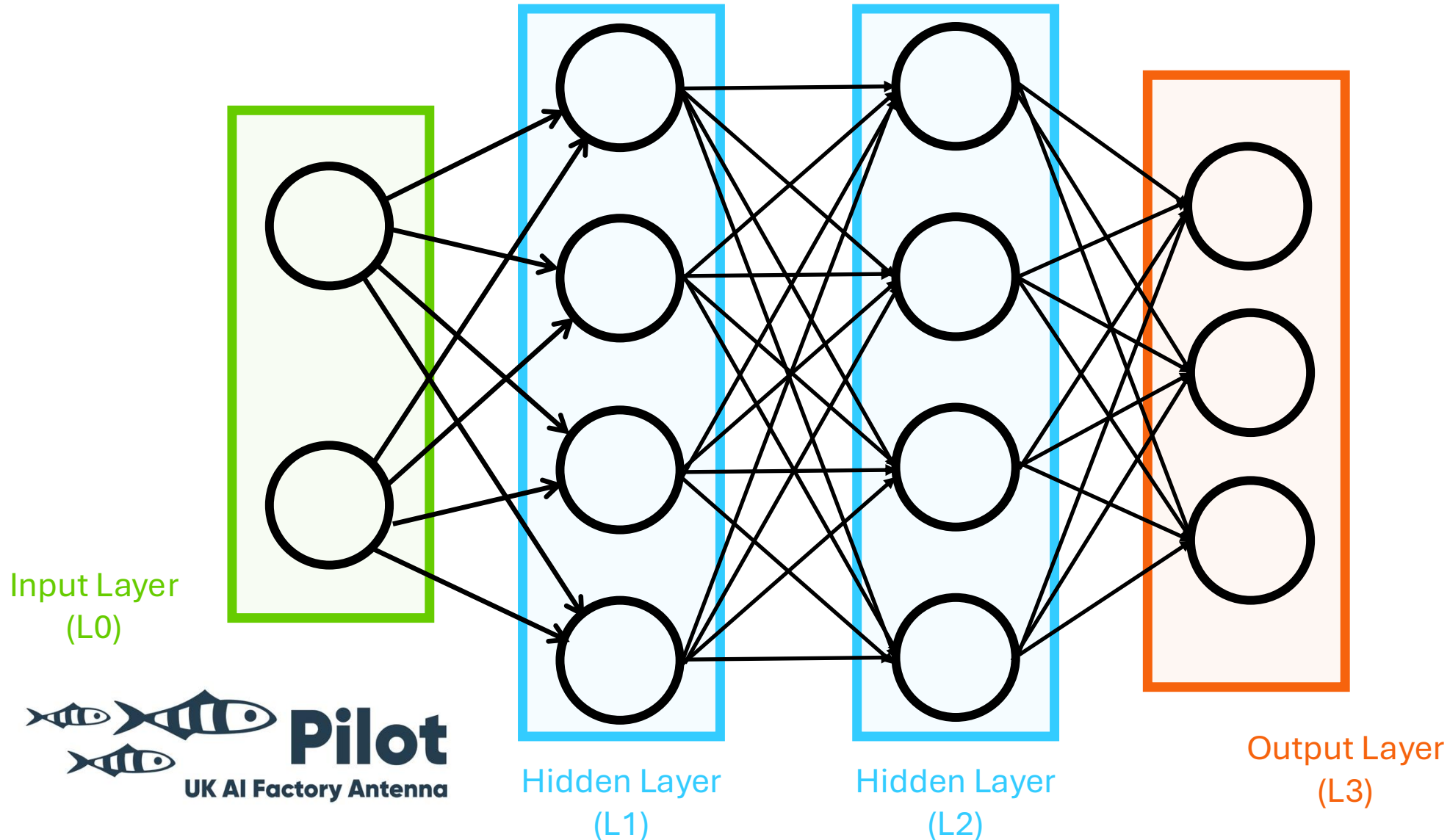
- “A neural network is a network or circuit of biological neurons, or, in a modern sense, an artificial neural network, composed of artificial neurons or nodes.” – Wikipedia
- “Neural networks, also known as artificial neural networks (ANNs) or simulated neural networks (SNNs), are a subset of machine learning and are at the heart of deep learning algorithms. Their name and structure are inspired by the human brain, mimicking the way that biological neurons signal to one another.” – IBM Cloud Education
- “A family of parametric, non-linear and hierarchical representation learning functions, which are massively optimized with stochastic gradient descent to encode domain knowledge, i.e. domain invariances, stationarity.” - Efstratios Gavves



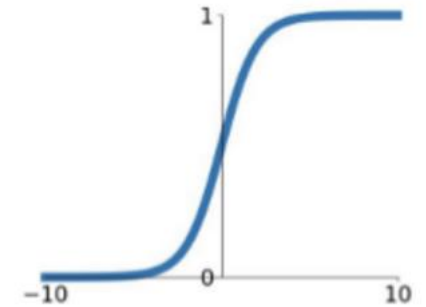
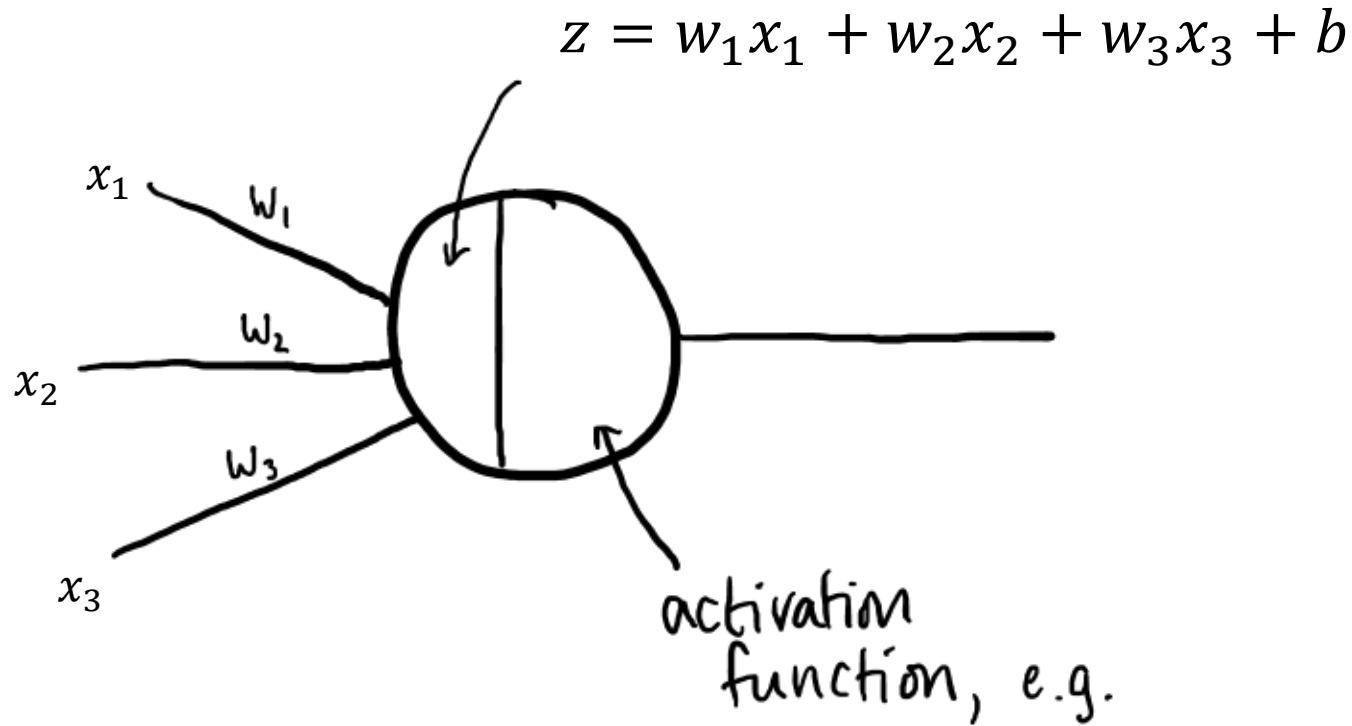
Components of a Neural Network



Components of a Neural Network



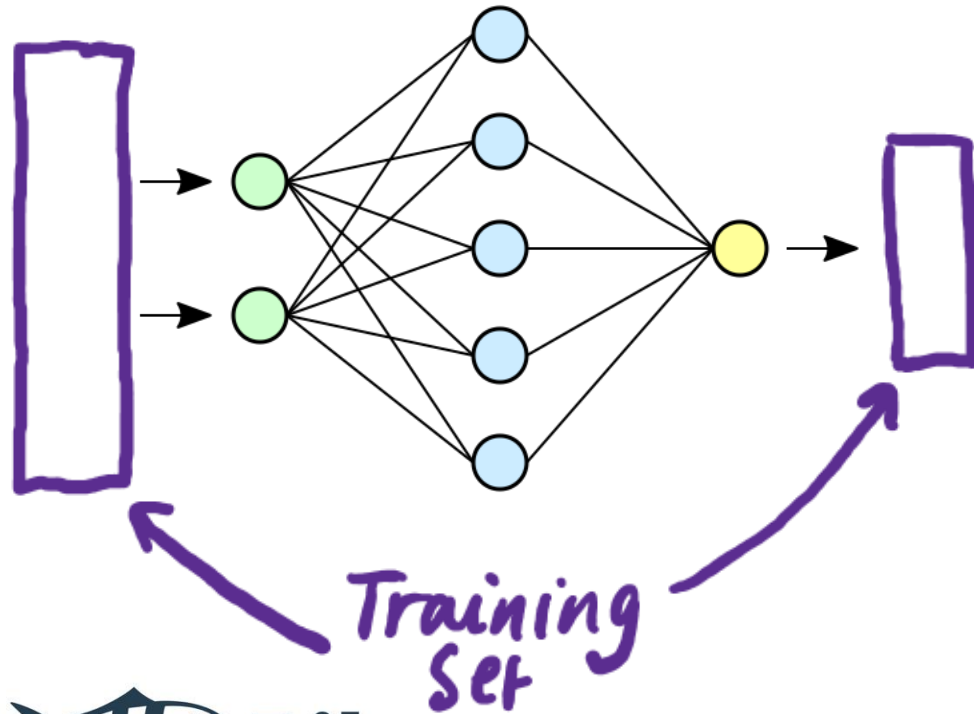
A “neuron”



$$a = \sigma(z) = \frac{1}{1 + e^{-z}}$$

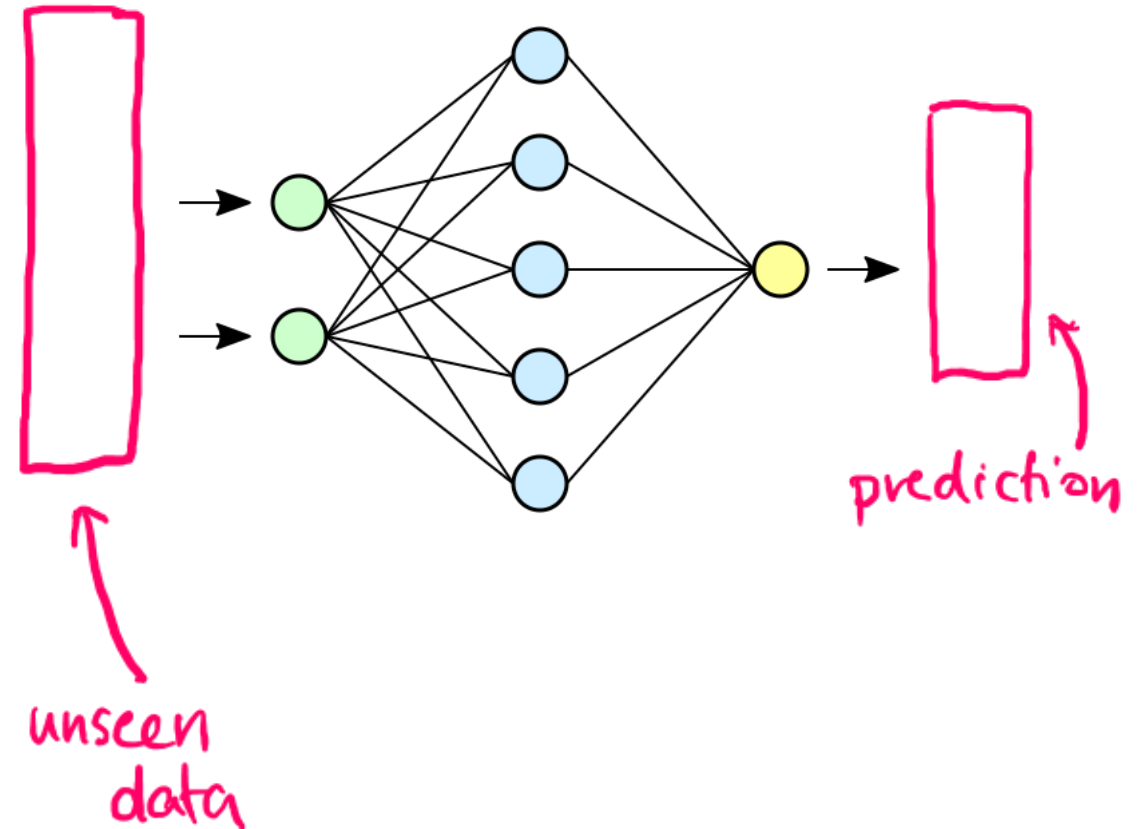
Training & Inference with a Neural Network

TRAINING



Training Set

PREDICTING

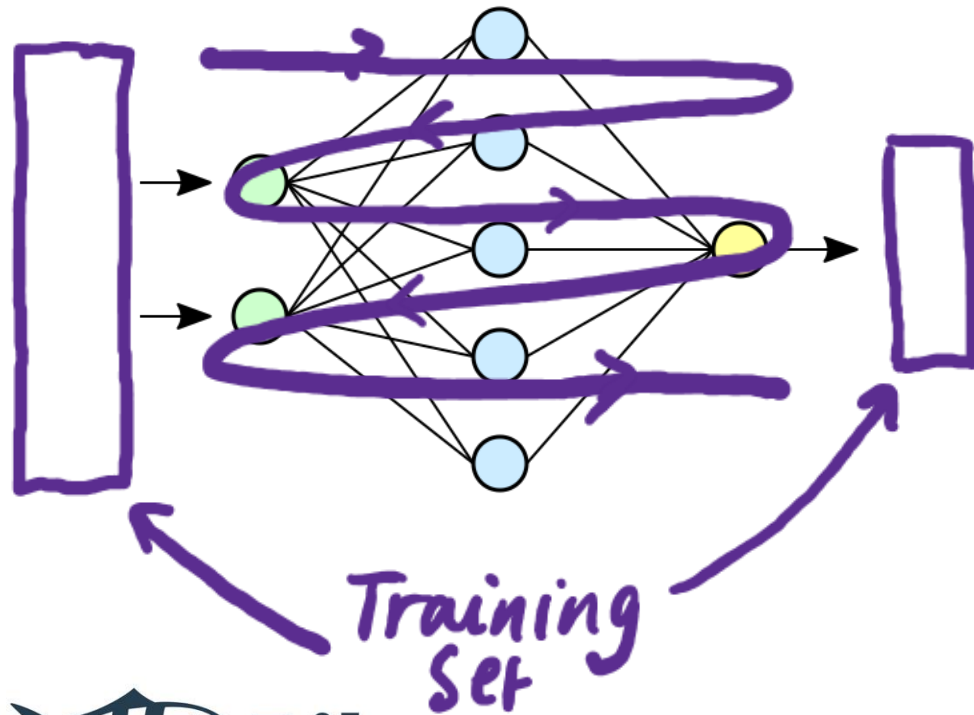


unseen data

prediction

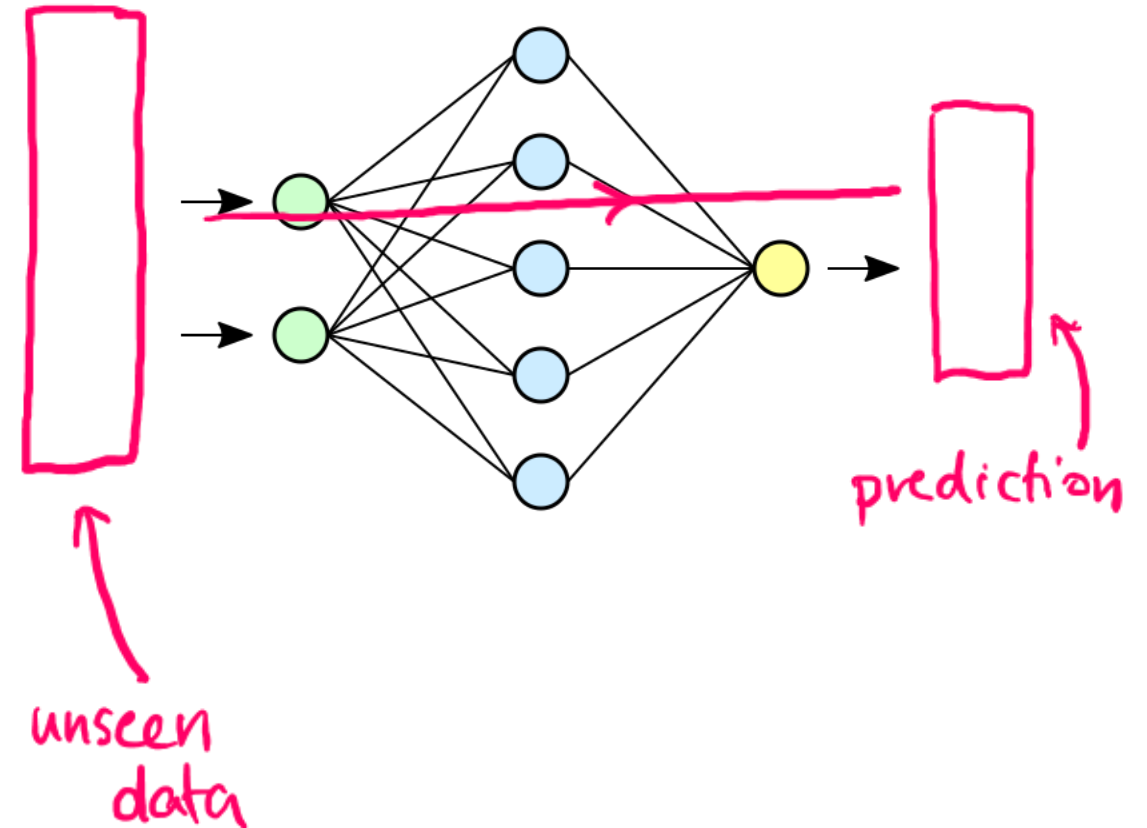
Training & Inference with a Neural Network

TRAINING



Training Set

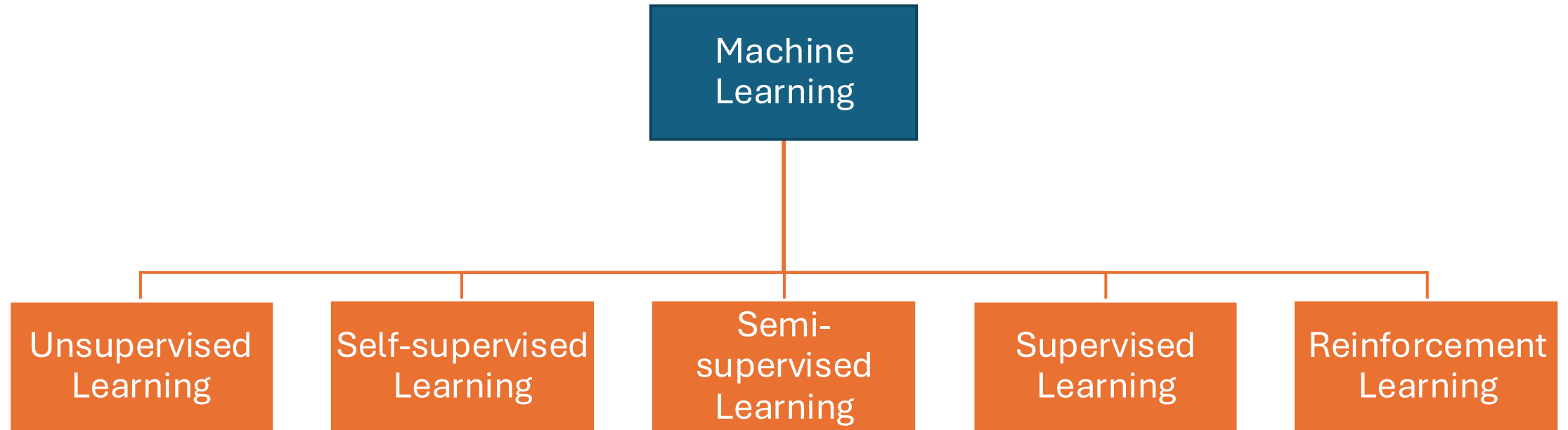
PREDICTING



unseen data

prediction

Practical: Neural Network



Feedback & Interest in Future Courses

- At the end of the course, we'll ask for your feedback and input that can help us shape our future course offering
- If you do happen to leave the course early, please still try to send feedback using the link available at the end of this presentation
- Also, feel free to speak to me at lunchtime with informal feedback or, particularly, future training needs for you or people in your organisation

09:30 **Welcome & Course Overview**

09:40 **What is AI? / Introduction to Machine Learning - Part 1**

10:30 *Break*

10:45 **Introduction to Machine Learning - Part 2**

11:35 **Introduction to Neural Networks**

12:30 *Lunch*

13:30 Discussion: **Ethical Considerations in AI**

14:00 **What happens when you enter a prompt into an AI assistant?**

15:00 *Break*

15:15 **Writing good prompts**

15:30 **Beyond the Prompt: Introduction to AI-assisted coding and Agentic AI**

16:20 **Wrap-Up**

16:30 *Close*

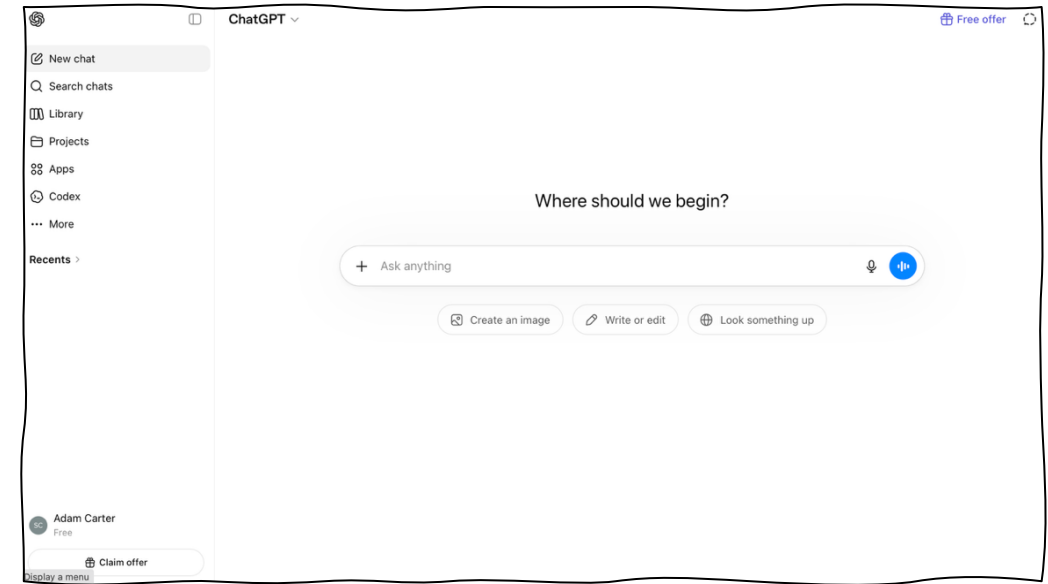
Course resumes at 13:30

Lunch



Discussion: Ethical Aspects of AI

- Environmental
- Risk of bias
- Respect for intellectual property rights
- Consolidation/centralisation of power for Big Tech
- Implications for labour forces
- Risk of cognitive offloading
- Disinformation & misinformation
- Creation of abusive imagery
- Mass surveillance
- Applications in warfare
- What concerns you most?
 - Why?
 - What can be done about it?
- Are any of these things that you had not considered or that you don't know what the issues relate to?
- 10 mins small group discussion followed by feedback and full group discussion



What happens when you use an AI assistant?

What happens when you enter your prompt?

- Step 1: Tokenisation:
 - Text must be converted into numbers to be passed through the Neural Network
 - The first step is to split up the full text into pieces called “tokens”

Tokenisation

- “your prompt is broken up into tokens” -> “your” “ prompt” “ is” “ broken” “ up” “ into” “ tokens” -> [4531, 10137, 374, 11234, 709, 1139, 11460]
- In simple terms, you can think of each word being a token
- Long or rare words can be split into multiple shorter tokens

Practical: Tokenisation

- For all:
 - Go to https://tiktokenizer.vercel.app/?model=cll00k_base and put some different messages into the box to see how they're tokenized
 - Notice that you can choose a tokenizer in the menu in the top right
- For those familiar with Python:
 - Follow through the notebook tokenisation.ipynb
 - If you select matching encodings, you should see that you get the same result as the web application linked above...

What happens when you enter your prompt?

- Step 1: Tokenisation
- Step 2: Embedding
 - Single ID numbers for each token are not enough as we want a representation that can, somehow, encode *meaning*

Embedding

- From the tokenisation stage, we have a long list of tokens
 - [4531, 10137, 374, 11234, 709, 1139, 11460]
- Each token is now converted into a vector
 - (vector = ordered list of numbers)

- Embedding Look Up:

- 4531 → [0.23, -1.17, ..., 0.88]

A long vector of numbers
(GPT-2 small uses a length of 768)

- The vector lives in a “high dimensional space”
 - Tokens with “similar meanings” end up “close together”

Embedding

- From the tokenisation stage, we have a long list of tokens
 - [4531, 10137, 374, 11234, 709, 1139, 11460]
- Each token is now converted into a vector
 - (vector = ordered list of numbers)

- Embedding Look Up:

- 10137 → [0.27, -0.19, ..., 1.34]

A long vector of numbers
(GPT-2 small uses a length of 768)

- The vector lives in a “high dimensional space”
 - Tokens with “similar meanings” end up “close together”

Practical: Embedding

- Follow through the notebook `embeddings.ipynb`

What happens when you enter your prompt?

- Step 1: Tokenisation
- Step 2: Embedding
- Step 3: The vectors are “stacked” into a stack of vectors

The vectors are then “stacked”

- “**your**” [0.23, -1.17, ..., 0.88]

$$\begin{pmatrix} 0.23 \\ -1.17 \\ \dots \\ 0.88 \end{pmatrix}$$

- “**prompt**” [0.27, -0.19, ..., 1.34]

$$\begin{pmatrix} 0.27 \\ -0.19 \\ \dots \\ 1.34 \end{pmatrix}$$

- ...

- “**tokens**” [-0.32, 0.42, ..., 2.07]

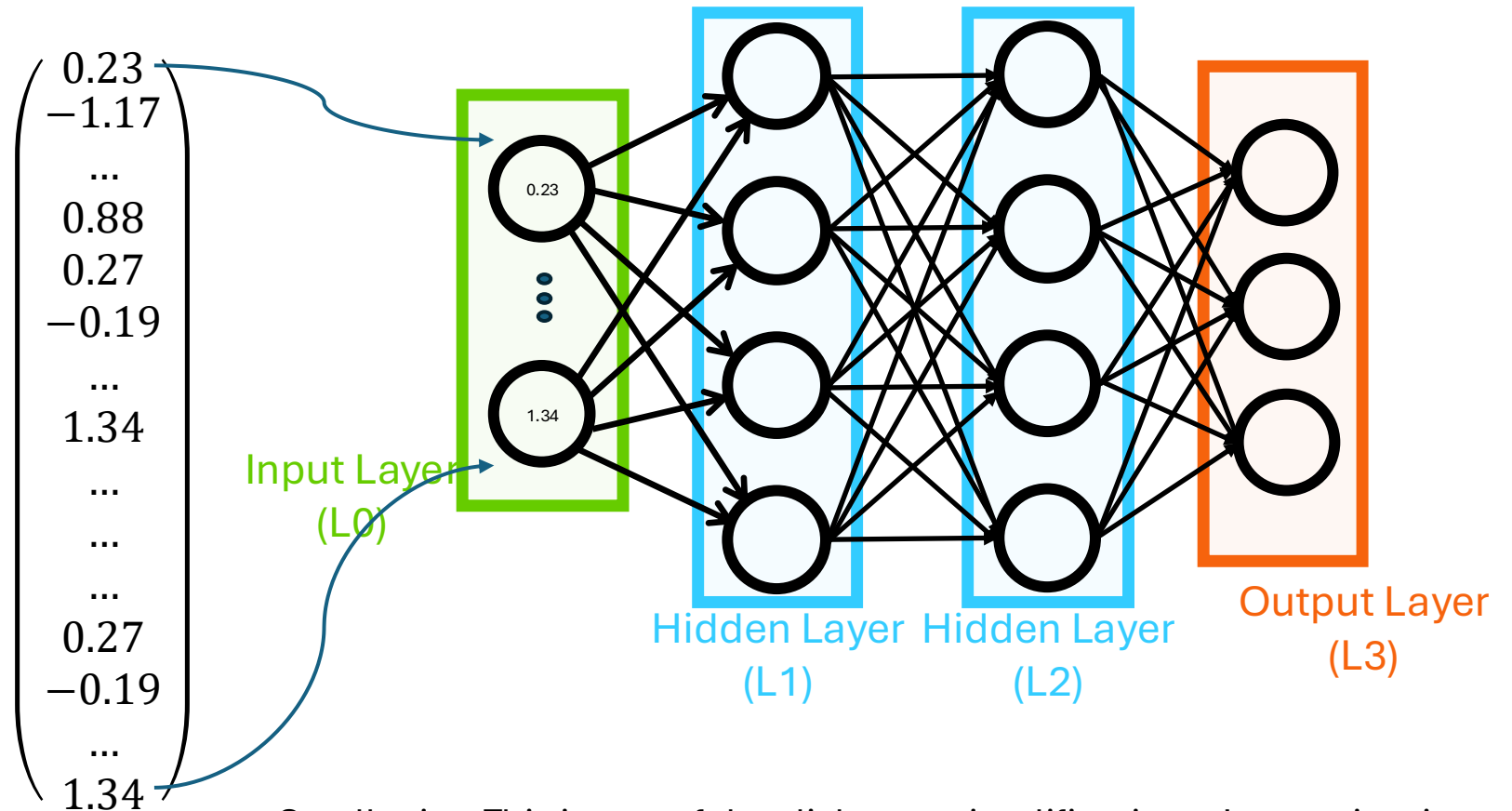
$$\begin{pmatrix} 0.27 \\ -0.19 \\ \dots \\ 1.34 \end{pmatrix}$$

$$\begin{pmatrix} 0.23 \\ -1.17 \\ \dots \\ 0.88 \\ 0.27 \\ -0.19 \\ \dots \\ 1.34 \\ \dots \\ \dots \\ \dots \\ 0.27 \\ -0.19 \\ \dots \\ 1.34 \end{pmatrix}$$

What happens when you enter your prompt?

- Step 1: Tokenisation
- Step 2: Embedding
- Step 3: The vectors are “stacked” into one long vector
- Step 4: This stacked vector is passed into the input layer of a neural network for a single “feed-forward” / inference step

This stacked vector is passed into the input layer of a neural network for a single “feed-forward” / inference step

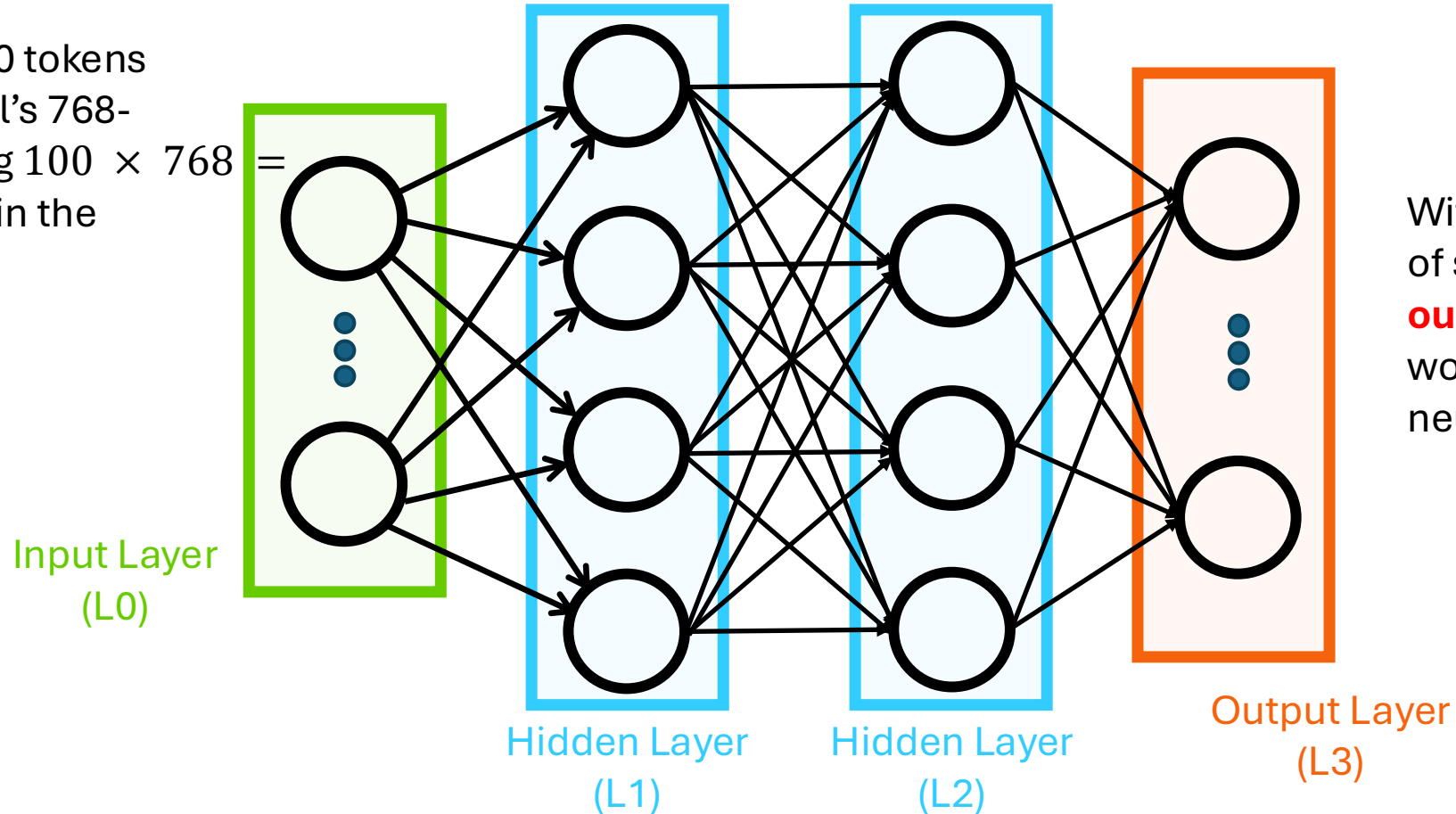


Small print: This is one of the slight oversimplifications: In practice, in a transformer you actually pass in a matrix. This is why the number of activations in the input layer doesn't change with context size.

What happens when you enter your prompt?

- Step 1: Tokenisation
- Step 2: Embedding
- Step 3: The vectors are “stacked” into one long vector
- Step 4: This stacked vector is passed into the input layer of a neural network for a single “feed-forward” / inference step
- Step 5: The neural network ultimately outputs another high-dimensional vector, this time with an entry associated with each possible output word
- Step 6: The word (token) with the highest value is chosen as the next word
- Step 7: If the output token is a special <END OF STREAM> token, the process stops, otherwise, this word is added to the prompt, and the process is repeated from step 2

A prompt with 100 tokens
using GPT-2 small's 768-
length embedding $100 \times 768 =$
76,800 neurons in the
input layer



With a dictionary
of size 50,257, the
output layer
would have **50,257**
neurons

A few additional details

0. The prompt is not just what you type in:

- It is combined with:
 - System instructions (invisible to the user)
 - *Every previous prompt and response in the “conversation”*
 - This **context** grows as the conversation goes on making things more computationally expensive/slow as the conversation grows

1. Tokenisation

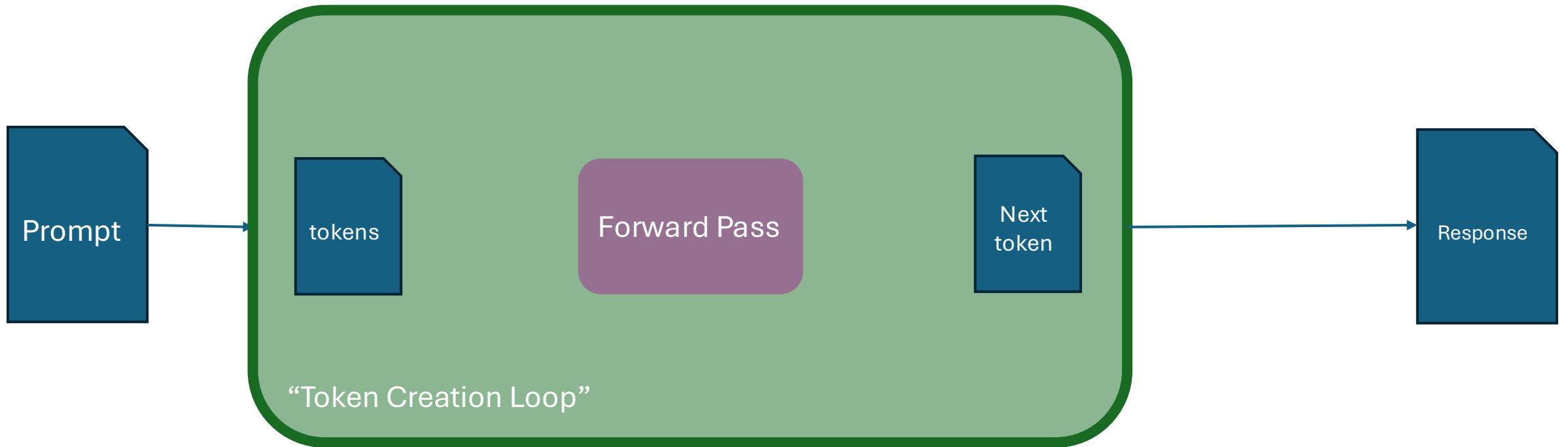
2. Embedding look up: 4531 $\rightarrow$ [0.23, -1.17, ..., 0.88]

2a. The embedding vector added to another vector that encodes position

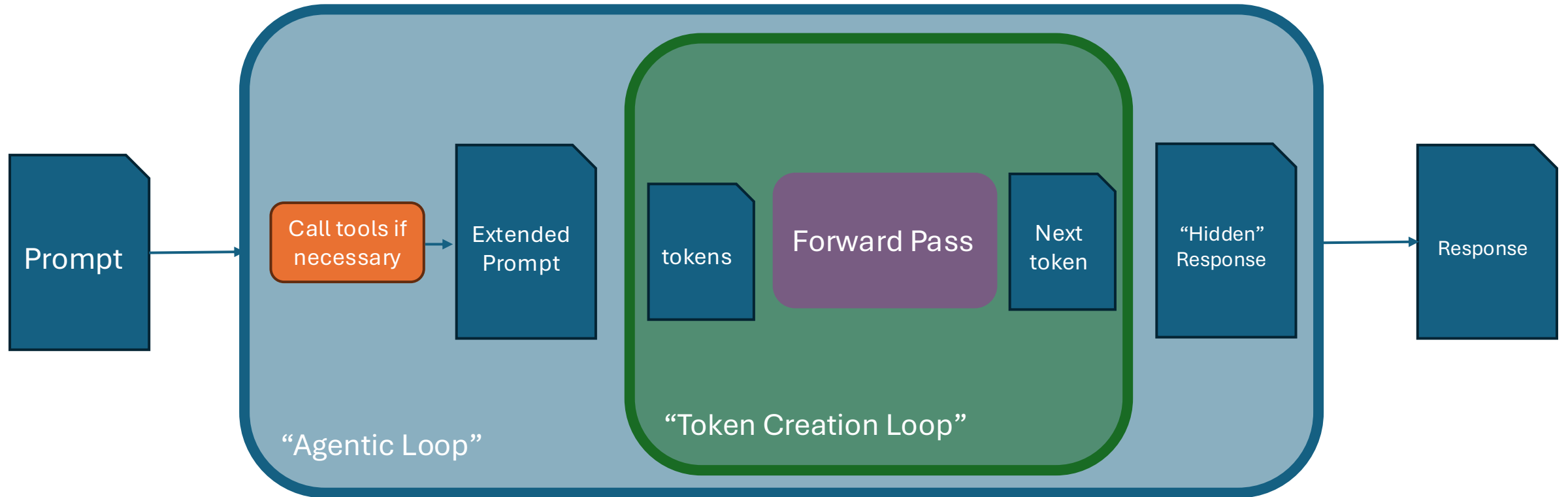
Some more additional details

- Sometimes the prompt is passed through a preprocessing pipeline (either with pattern matching or being passed to a smaller routing model) to help with:
 - Safety / moderation / policy enforcement
 - Selection of models (e.g. to decide to use a more complex model or whether a simple model would be sufficient to answer the question)
 - Tool calling (e.g., do I need to perform a web search to address a question in the prompt?)
- Current products have started to adopt strategies used in agentic AI systems:
 - They can loop several times (with hidden outputs) to perform multiple “reasoning” steps, calling tools in intermediate stages (e.g., to run python code)

From simple chat...

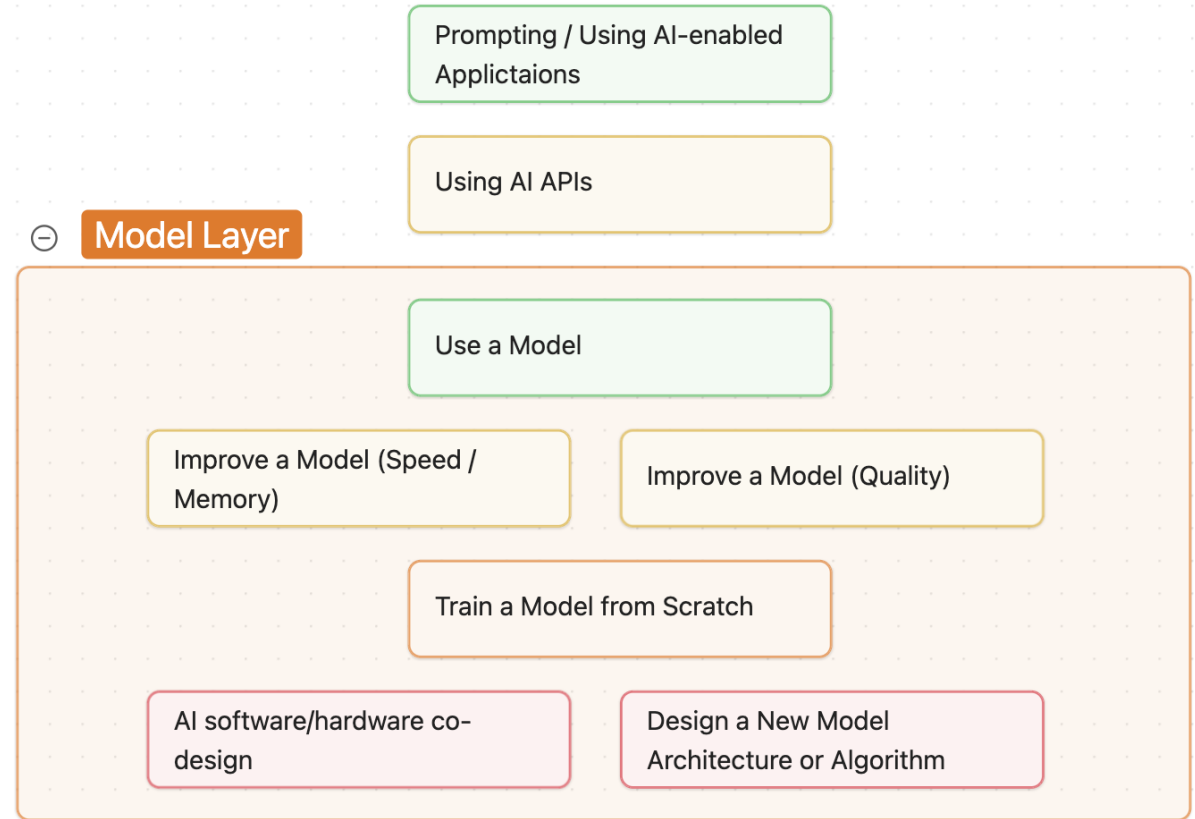


... to Agentic AI



A brief aside...

- How do people “use” AI?
- Where can I get models?
 - <https://huggingface.co>
 - <https://lmstudio.ai>
 - <https://www.kaggle.com/models>
 - ...or from within frameworks, e.g., PyTorch, TensorFlow



Break

- Course resumes at 15:15

Writing good prompts

Writing Good Prompts

- ...is about providing enough context
- Remember that you can work iteratively, because a conversation remembers earlier steps,
- but if you want to get straight to a good answer, give the model context

Not so good...

- Explain neural networks

Better...

- You are training a general audience. Explain backpropagation intuitively and then provide some basic code examples which illustrate the ideas. Include a worked example and finish with three misconceptions that students commonly have.

You don't just have to ask questions

- You can say things like:
 - Test me on...
 - Come up with some questions on ... and evaluate my answers
 - I'd like you to teach me about One step at a time. Use a socratic approach where you ask me leading/guiding questions.
 - Play the role of ...
 - Coach me on...
 - Critique the following:...
 - Help me create...

Writing reusable prompts

- ...which could be incorporated into code, a script, a workflow or a product
- For prompts that are to be used repeatedly, you want them to be good, and generalizable
 - Prompt engineering
 - Iterative testing, evaluating and improvement
 - Give examples (if you give 3 examples, this is sometimes called a “three shot prompt”)
 - Use template-like notation to help identify key inputs and output format
 - Remember that “sanitisation” of user-input is important, and difficult

Beyond the Prompt

- “I'd like to create a standalone webapp that can be accessed locally only from my machine. The app should implement Conway's Game of Life on a 10 by 10 grid allowing the user to turn cells on and off before letting the user start the simulation.”
- DEMO

Final Thoughts

- AI is a broad, fast-moving field
 - There is much that is still to be found out
- We should not lose sight of the many ethical concerns associated with AI
- Although there is (too much?) hype around AI, there is also great potential for application in many domains
- In Pilot-UKAIFA we're keen to target future training in areas that can help people. We'd very much like your input!

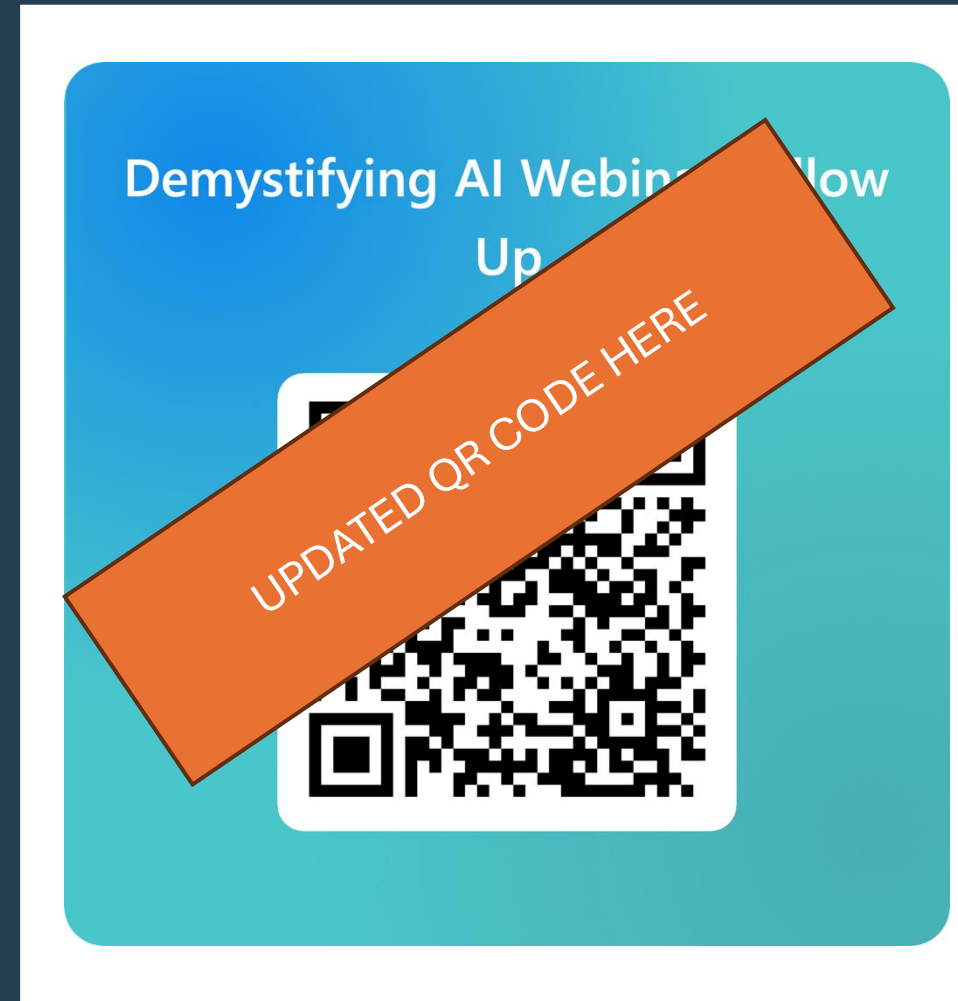
Keep in Touch with Pilot - UKAIFA

- If you're interested in our future training courses or the wider project, please complete the following short form:

<URL

UPDATED URL HERE

- You can also use this form to provide feedback on today's webinar. We're always happy to receive feedback.
- Find out more about the project at:
www.pilot-ukaifa.ai



Acknowledgements and Reuse

- Most of this presentation has been created by Adam Carter, EPCC, The University of Edinburgh as part of the Pilot-UKAIFA project
 - It draws some elements for courses previously delivered on EPCC's *MSc in High Performance Computing with Data Science* with some slide content originally developed by others at EPCC including Ally Hume
- © 2026 The University of Edinburgh
 - You are welcome to reuse this material under the terms of CC BY 4.0